A Scalable AI-Enabled Framework for Near-Real-Time Social Trend Detection from Multi-Source Vietnamese Online Content

Trong-Hop Do[1,2], Kiet G.Quach[1,2], Long T.Pham[1,2], Anh Thu T.Huynh[1,2], Nhut Hoang Trang[1,2], Qui Ngoc Nguyen[1,2], Thanh-Tuyen T.Do[1,2,*]
[1]University of Information Technology
[2]Vietnam National University, Ho Chi Minh City, Vietnam
Email: {23520819, 23520887, 23521535, 18520123}@gm.uit.edu.vn
{hopdt, quinn, tuyendtt}@uit.edu.vn
*Corresponding author.

***Abstract*—The rapid growth of TikTok, YouTube, and online news platforms has generated a large amount of online content, which contains many social trends that can spread quickly. However, detecting trends from online data remains challenging because the data are heterogeneous, noisy, and existing tools often focus on individual platforms. This paper proposes a framework for near-real-time social trend detection and analysis from Vietnamese online content. The system uses Kafka for data ingestion, PySpark for batch processing, embedding models for semantic representation, HDBSCAN for clustering related content, and large language models for extracting topics, summaries, sentiment, risk levels, and recommendations. A trend deduplication mechanism is also used to reduce duplicate trends across processing batches. Preliminary experiments on 5,305 input items show that the system can detect 25 trend clusters and provide additional analyses to support digital trend monitoring. The initial results suggest that combining Kafka, Spark, embeddings, HDBSCAN, and large language models is a feasible approach for detecting social trends from Vietnamese online content.**

***Index Terms*—social trend detection, Vietnamese online content, social media analytics, Kafka, Spark, HDBSCAN, large language models**

I. Introduction

The rapid growth of online platforms has changed the way information is created, shared, and consumed. In Vietnam, the number of social media users reached approximately 76.2 million in early 2025, showing that social media has become an important data source for monitoring user behavior, events, and social trends [1]. Platforms such as TikTok, YouTube, and online news sites continuously generate a large volume of content in different formats, including short videos, titles, descriptions, hashtags, and news articles. Within these data streams, many topics can quickly develop into trends, affecting communication, marketing, content management, and organizational decision-making.

The problem of trend detection is closely related to Topic Detection and Tracking (TDT), which aims to identify newly emerging topics and track their development over time [2]. Previous studies have used social media data, especially Twitter, to detect emerging topics based on term frequency, temporal patterns, and the social role of users [3], [4]. However, many existing methods focus on a single platform, while social trends today can spread across multiple online sources. In addition, online content is often short, noisy, heterogeneous, and changes rapidly, which requires a scalable and automated processing pipeline [5].

Recent methods for semantic representation provide a more effective way to group related content. Sentence-BERT has shown that sentence embeddings can be effectively used for semantic search and text clustering tasks [6]. In addition, HDBSCAN is a density-based clustering algorithm that does not require the number of clusters to be specified in advance and can handle data with different densities [7]. More recently, large language models (LLMs) have been widely used for text analysis tasks such as summarization, sentiment classification, and information extraction. However, their performance still depends on the data, prompt design, and application context [8].

In this study, we propose a framework for near-real-time social trend detection and analysis from multi-source Vietnamese online content. The system uses Apache Kafka for data ingestion, PySpark for batch processing, embedding models to represent content as semantic vectors, and HDBSCAN to detect clusters of related content. Then, an LLM is used to extract topics, generate summaries, analyze sentiment, assess risk levels, and provide user-oriented recommendations. The main contributions of this study are summarized as follows:

- We build a multi-source data processing pipeline for Vietnamese social trend detection.
- We combine semantic representation and density-based clustering to detect trends without predefining the number of clusters.
- We integrate an LLM to provide interpretable trend-level analysis, including topic extraction, summarization, sentiment analysis, risk assessment, and user-oriented recommendations.

II. Related Work

Trend detection is closely related to Topic Detection and Tracking (TDT), where the goal is to detect new topics or events in a text stream and monitor their development over time [2]. Kleinberg [9] proposed a burst detection model to identify periods when a topic or keyword suddenly increases in frequency. In the social media context, Cataldi et al. [3] proposed a method for detecting emerging topics on Twitter by considering temporal frequency and the social role of users. Aiello et al. [4] further studied trending topic detection on Twitter and showed that social media data are often short, noisy, dynamic, and require automatic processing methods. However, most of these studies focus on a single platform, especially Twitter, while current social trends may spread across many platforms such as TikTok, YouTube, and online news. Social media analytics is also an important research direction for studying user opinions, behaviors, and trends on online platforms. Batrinca and Treleaven [5] reviewed techniques for social media analytics, including data collection, data cleaning, content analysis, and visualization. They also emphasized the need for scalable pipelines to process large and noisy data. In addition, traditional topic modeling methods have been widely used to discover topics from unlabeled text collections [10]. However, methods such as LDA may have limitations when dealing with short, informal, and context-rich social media text. This motivates the use of modern semantic representations.

A common recent approach is to represent text as semantic vectors before clustering. Reimers and Gurevych [6] proposed Sentence-BERT to produce sentence embeddings that can be directly compared using cosine similarity. This representation is useful for semantic search, clustering, and topic discovery. After obtaining semantic embeddings, clustering algorithms such as HDBSCAN can be used to group related content. Campello et al. [7] introduced HDBSCAN as a hierarchical density-based clustering method that can detect clusters with different densities and identify noise. Compared with K-means, HDBSCAN does not require the number of clusters to be predefined; compared with DBSCAN, it is more stable when the data have non-uniform density [7], [11]. BERTopic [12] also shows that combining transformer-based embeddings, dimensionality reduction, clustering, and topic representation can produce interpretable topics. More recently, Jiang et al. [13] showed that combining embeddings with temporal information can improve topic detection and tracking.

Beyond detecting trend clusters, practical trend monitoring systems should provide interpretable information, such as summaries, sentiment, risk levels, and recommendations. Hanny et al. [14] studied the combination of pre-trained language models and clustering for joint topic-sentiment modeling in social media short texts. The development of LLMs has further enabled the automation of text analysis tasks, including sentiment classification, summarization, and information extraction. Zhang et al. [8] reviewed the use of LLMs for sentiment analysis and showed that LLMs can handle subjective tasks flexibly, although their performance depends on the task complexity, data, and prompt design.

Based on these studies, we identify three main research gaps. First, many trend detection methods still focus on a single platform instead of multi-source online data [3], [4]. Second, topic modeling or clustering methods usually detect topic groups but do not provide enough additional analysis, such as summaries, sentiment, risk levels, and actionable recommendations [10], [12]. Third, the integration of LLMs into a near-real-time big data pipeline for multi-source trend detection and analysis has not been fully explored [8], [14]. To address these gaps, this paper proposes a framework that combines Kafka, Spark, semantic embeddings, HDBSCAN, and LLMs to detect, merge, and analyze social trends from Vietnamese online content.

III. Proposed Method

A. System Overview

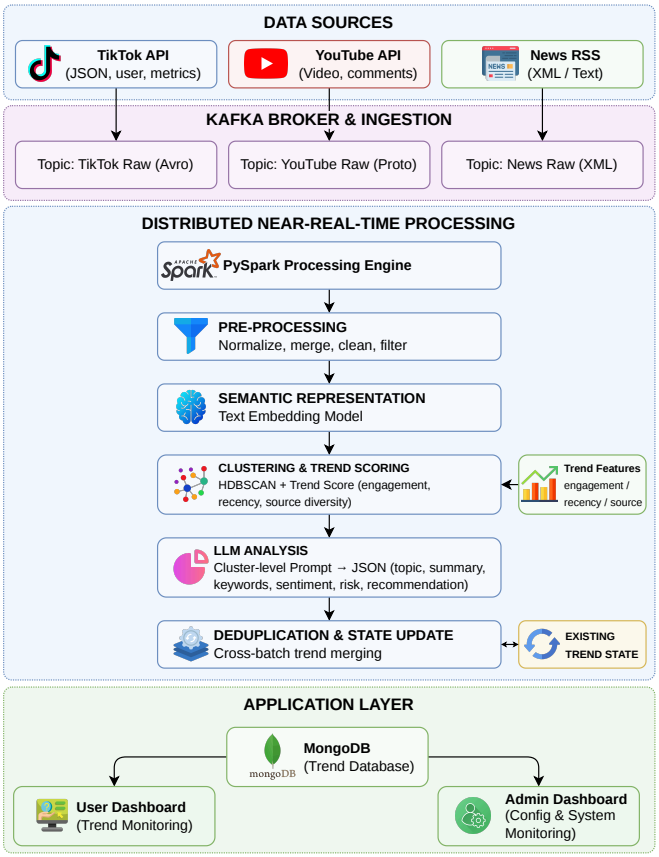


Figure 1. Overall architecture of the proposed near-real-time social trend detection framework.

The system is designed to support near-real-time detection and analysis of social trends from multiple Vietnamese online content sources, including TikTok, YouTube, and online news. The input of the system consists of raw content collected periodically from these platforms, including titles, descriptions, textual content, hashtags, posting time, data source, and engagement signals such as views, likes, comments, and shares. The output is a list of analyzed trends, where each

trend includes a topic, summary, keywords, sentiment, risk level, content type, trend score, and representative items.

Fig. 1 illustrates the overall architecture of the proposed system. The processing pipeline contains four main stages: data preprocessing, semantic representation and clustering, trend analysis using a large language model, and trend deduplication across consecutive processing batches. The system architecture uses Apache Kafka for data ingestion [15], PySpark for batch processing [16], HDBSCAN for density-based clustering [11], a large language model for trend-level analysis [8], and MongoDB for storing trend outputs [17].

B. Preprocessing and Input Representation

Each collected item is normalized into a unified schema:

$$x_i = \{id, text, source, timestamp, metrics\}, \tag{1}$$

where $text$ includes the title, description, content, and hashtags, while $metrics$ includes views, likes, comments, and shares.

The input text for the embedding model is constructed as:

$$text_i = title_i \oplus description_i \oplus hashtags_i, \tag{2}$$

where $\oplus$ denotes text concatenation.

Items with very short text, missing information, or noisy content are removed. The system also applies quality filtering based on engagement signals, such as the minimum engagement rate, number of views, or the number of followers if available. This step aims to reduce noise before clustering, since social media data often contain spam, repeated content, and text with limited semantic value.

Each preprocessed item is converted into a semantic vector using an embedding model:

$$e_i = Embedding(text_i). \tag{3}$$

Here, e_i is the semantic representation of the input item x_i. These vectors are used as the input for the clustering algorithm. Semantic representation is motivated by prior work showing that sentence embeddings are useful for semantic search and text clustering tasks [6].

C. Trend Cluster Detection Using HDBSCAN

Given a set of embeddings $E = \{e_1, e_2, ..., e_n\}$, the system applies HDBSCAN to group semantically related content. HDBSCAN is selected because it does not require the number of clusters to be predefined and is suitable for social media data, where the number and size of trends may change across batches [7], [11].

The output of HDBSCAN is a set of cluster labels:

$$C = \mathrm{HDBSCAN}(E, m), \tag{4}$$

where m denotes the minimum cluster size.

Each cluster c_j is considered a trend candidate if the number of items in the cluster is greater than a predefined minimum threshold. Items labeled as noise can be reassigned to the nearest cluster if their cosine distance is smaller than a predefined threshold. This step helps reduce the noise rate while maintaining the semantic consistency of the clusters.

D. Trend Scoring

To rank the detected trend clusters, the system computes a trend score based on three main factors: engagement, recency, and source diversity. For each cluster c_j, the trend score is defined as:

$$Score(c_j) = \alpha E_j + \beta R_j + \gamma D_j. \tag{5}$$

Here, E_j is the engagement score of the cluster, which is computed from views, likes, comments, and shares. R_j is the recency score, which gives higher priority to content that appears closer to the processing time. D_j is the source diversity score, which reflects whether the trend appears across one or multiple platforms. The parameters α, β, and γ are weights that can be adjusted according to the system objective.

The engagement score is computed as:

$$\begin{aligned} E_j = &\log(1 + views_j) + \lambda_1 \log(1 + likes_j) \\ &+ \lambda_2 \log(1 + comments_j) \\ &+ \lambda_3 \log(1 + shares_j). \end{aligned} \tag{6}$$

The logarithmic function is used to reduce the effect of extremely large view counts, which may otherwise dominate the trend score. The recency score R_j is computed based on the time gap between the processing time and the average posting time of the items in the cluster. The source diversity score D_j increases when a cluster contains content from multiple platforms.

E. Trend Analysis Using a Large Language Model

After detecting trend clusters, the system selects representative items from each cluster based on engagement scores. Instead of analyzing each item separately, the system performs analysis at the cluster level to reduce processing cost and produce a more summarized output. The input to the LLM includes titles, short descriptions, hashtags, data sources, and engagement statistics of the representative items.

The LLM is instructed to return a structured JSON output with seven fields: topic, summary, keywords, sentiment, content type, risk level, and recommendation. The output is checked before storage. If the response is invalid or missing important fields, the system can call the model again or use default values. The use of LLMs for text analysis is motivated by recent studies showing their ability to support sentiment analysis, summarization, and information extraction [8].

F. Trend Deduplication Across Processing Batches

Since the system processes data in batches, the same trend may appear in multiple processing rounds. Therefore, a deduplication mechanism is used to merge a newly detected trend with an existing trend in the database when they refer to the same topic or event.

Given a new trend t_{new} and an existing trend t_{old}, the system computes an overall similarity score:

$$\begin{aligned} Sim(t_{new}, t_{old}) = &w_1 S_{emb} + w_2 S_{key} \\ &+ w_3 S_{time} + w_4 S_{source}. \end{aligned} \tag{7}$$

Here, S_{emb} is the cosine similarity between the representative embeddings of the two trends. S_{key} is the keyword overlap, which can be computed using Jaccard similarity. S_{time} measures the temporal closeness between the two trends, while S_{source} measures the similarity of their source distributions. The parameters w_1, w_2, w_3, and w_4 are the weights of the corresponding components.

If the similarity score is greater than a threshold θ, the new trend is merged into the existing trend. The system then updates the number of items, total engagement, keywords, representative items, update time, and trend score. Otherwise, the system creates a new trend record in the database. Algorithm 1 summarizes the cross-batch trend deduplication procedure.

Algorithm 1 Cross-Batch Trend Deduplication

Require: t_{new}, existing trends T, threshold θ
Ensure: Updated trend database
for each $t_{old} \in T$ **do**
    Compute $Sim(t_{new}, t_{old})$
end for
Select t^* with the highest similarity score
if $Sim(t_{new}, t^*) \geq \theta$ **then**
    Merge t_{new} into t^*
    Update statistics, keywords, items, and score
else
    Insert t_{new} as a new trend
end if

This procedure helps reduce duplicate trends across batches and supports tracking the development of the same trend over time.

G. System Output

After the full pipeline, each trend is stored as a document in MongoDB [17]. Each trend record includes the trend ID, topic, summary, keywords, sentiment, content type, risk level, recommendation, trend score, source distribution, representative items, creation time, and update time. These results are used for dashboard visualization, allowing users to monitor, search, filter, and evaluate emerging trends in Vietnamese online content.

IV. Experiments and Results

A. Experimental Setup

The experiment was conducted on Vietnamese online content collected from three sources supported by the system, including TikTok, YouTube, and online news. Each data item contains main fields such as title, description, hashtags, data source, posting time, and engagement signals, including views, likes, comments, and shares.

The system was evaluated on two consecutive data batches with a total of 5,305 input items. After cleaning and quality filtering, 4,679 items remained and were used for semantic representation, clustering, and trend analysis. Specifically, Batch 1 contained 2,024 raw items and Batch 2 contained 3,281 raw items. After quality filtering, the remaining numbers of items were 1,875 and 2,804, respectively. Table I summarizes the number of items at each processing stage.

The experimental pipeline includes the following steps: normalizing data into a unified schema, cleaning text, filtering low-quality content, converting text into semantic vectors, clustering with HDBSCAN, analyzing trend clusters using a large language model, checking duplicate trends across batches, and storing the results. Each cluster is considered a trend candidate if the number of items in the cluster is greater than a predefined minimum threshold. For each detected trend, the system generates several fields, including topic, summary, keywords, sentiment, content type, risk level, trend score, and representative items.

Table I
Statistics of the Experimental Data

Processing Stage	Batch 1	Batch 2	Total
Raw items	2,024	3,281	5,305
After cleaning	2,022	3,241	5,263
After filtering	1,875	2,804	4,679
Filtering rate	92.6%	85.5%	88.2%

B. Trend Cluster Detection Results

Table II
Clustering Results of the Proposed Method

Metric	Batch 1	Batch 2	Total
Clusters detected	12	13	25
Avg. cluster size	135.4	157.7	147.0
Min cluster size	40	58	40
Max cluster size	260	292	292
Noise rate	0.0%	0.0%	0.0%

Table II presents the HDBSCAN clustering results on the two data batches. The system detected 25 trend clusters in total, including 12 clusters from Batch 1 and 13 clusters from Batch 2. The average cluster size was 147 items, with the smallest cluster containing 40 items and the largest cluster containing 292 items. After the second-pass assignment step, the noise rate reached 0.0%, indicating that reassigning noise items to the nearest cluster helped improve the coverage of the pipeline.

These results show that the number of detected clusters was relatively stable across the two batches, although Batch 2 contained more input data than Batch 1. The average cluster size was also relatively large, suggesting that the detected trends were not only small isolated groups but also had a certain level of spread within the dataset. However, the 0.0% noise rate should be interpreted with caution, because it was obtained after the second-pass reassignment step. This step improves data coverage by assigning noise items to the nearest cluster when their cosine distance is below a predefined threshold. However, it may also introduce the risk of assigning weakly related items to existing clusters. Therefore, future work should further evaluate the impact of reassignment thresholds on cluster purity and topic coherence.

C. Cross-Batch Trend Deduplication Results

Table III presents the results of the trend deduplication mechanism across the two processing batches. Batch 1 generated 12 new trends, while Batch 2 generated 13 new trends. No trend in Batch 2 was merged with an existing trend from Batch 1, resulting in a merge rate of 0%.

Table III
CROSS-BATCH TREND DEDUPLICATION RESULTS

Metric	Batch 1	Batch 2
New trends	12	13
Merged trends	0	0
Total trends saved	12	13
Merge rate	0%	0%

The merge rate of 0% does not necessarily indicate that the deduplication mechanism is ineffective. In this experiment, the two batches were collected using different keyword settings, which produced different trend topics. As a result, no detected trend in Batch 2 satisfied the similarity threshold for merging with trends from Batch 1. A more complete evaluation of the deduplication mechanism requires longer-term data collection with overlapping topics across multiple batches. However, this mechanism remains important in the system architecture because it allows the system to check duplicate trends before storage and supports tracking the development of the same trend across consecutive processing batches.

D. Preliminary Manual Evaluation

To examine the practical meaning of the detected trend clusters, we conducted a preliminary manual evaluation on 15 detected trends. A trend was considered valid if it had a clear topic, its representative items were semantically related, and it showed meaningful engagement signals. The result shows that 14 out of 15 trends were considered valid, corresponding to a trend correctness score of 93.3%.

Table IV
PRELIMINARY MANUAL EVALUATION

Criterion	Evaluation Method	Result
Trend correctness	Valid / Invalid	93.3%
Topic coherence	Representative items	Preliminary review
Duplicate detection	Potential duplicate trends	Preliminary review

The preliminary manual evaluation in Table IV provides initial evidence that most of the detected clusters are meaningful in practice. However, since only 15 trends were manually reviewed and the evaluation did not involve multiple independent annotators, the result should be interpreted as preliminary. Future work should expand the number of evaluated trends, involve multiple annotators, and report inter-annotator agreement to improve the reliability of the evaluation.

E. Data Source Distribution

Table V presents the distribution of data sources in the detected trend clusters. The results show that the detected trends were mainly formed from YouTube and TikTok content. YouTube contributed 2,882 items, accounting for 78.5%, while TikTok contributed 791 items, accounting for 21.5%. News did not contribute any item to the detected trend clusters.

These results show that, although the system was designed to support multiple data sources, the current experiment mainly validates the framework on two video-based platforms, YouTube and TikTok. The absence of News items in the detected clusters may be caused by several factors, such as keyword mismatch in RSS feeds, limited availability of news data during collection, or the need for source-specific preprocessing strategies for online news articles. Therefore, the current results should not be interpreted as complete evidence of multi-source trend detection. Instead, they should be viewed as an initial validation on Vietnamese online content, mainly from video-based sources.

Table V
DATA SOURCE DISTRIBUTION IN DETECTED TREND CLUSTERS

Source	Items	Ratio	Avg./Trend
YouTube	2,882	78.5%	115.3
TikTok	791	21.5%	31.6
News	0	0.0%	0
Total	3,673	100%	146.9

F. Distribution of Trend Analysis Results

Table VI
DISTRIBUTION OF TREND ANALYSIS RESULTS

Analysis Type	Category	Count	Ratio
Sentiment	Positive	19	76%
	Neutral	5	20%
	Negative	1	4%
Risk level	Safe	19	76%
	Cautious	5	20%
	High risk	1	4%
Content type	Entertainment	15	60%
	Commercial	7	28%
	Drama	2	8%
	Social issue	1	4%

In addition to detecting trend clusters, the system also generates additional analyses for each trend using a large language model. Table VI summarizes the distribution of these analysis results. Among the 25 detected trends, 19 trends were classified as positive, 5 as neutral, and 1 as negative. Regarding risk level, the system identified 19 trends as safe, 5 as cautious, and 1 as high risk. For content type, entertainment accounted for the largest proportion with 60%, followed by commercial content with 28%, drama with 8%, and social issues with 4%.

These results show that the large language model can provide additional interpretable information for the detected trend clusters. This helps users quickly understand the main characteristics of each trend. However, these distributions mainly reflect the output of the LLM and should not be considered the primary evidence for the effectiveness of trend detection. Their main role is to support the interpretation of the detected clusters.

G. Functional Comparison with Existing Platforms

In addition to the experimental results, we compare the main functions of the proposed system with several popular platforms, including Google Trends and TikTok Creative Center. Here, GT denotes Google Trends, and TCC denotes TikTok Creative Center. Table VII shows that existing platforms

Table VII
FUNCTIONAL COMPARISON WITH EXISTING PLATFORMS

Feature	GT	TCC	Proposed
Data source	Search	TikTok	Multi-source design
Auto discovery	Limited	Specific	Yes
Semantic clustering	No	No	Yes
Sentiment/risk	No	Limited	Yes
Cross-batch dedup.	No	No	Yes
Dashboard	Yes	Yes	Yes

mainly focus on a specific data source, while the proposed system is designed to support data collection and analysis from multiple sources. However, it should be noted that in the current experiment, the system was mainly validated on YouTube and TikTok data, while News did not contribute items to the detected trend clusters.

The comparison shows that the main difference of the proposed system lies in its ability to detect trends through semantic clustering, provide additional analysis using a large language model, and reduce duplicate trends across processing batches. While Google Trends mainly reflects public interest through search data, TikTok Creative Center focuses on trends within the TikTok platform. In contrast, the proposed system aims to provide a more flexible pipeline for collecting, clustering, and interpreting trends from Vietnamese online content.

Overall, the preliminary experimental results show that the system can process Vietnamese online data, detect meaningful trend clusters, and generate additional analyses to support digital trend monitoring. Combining semantic embeddings with HDBSCAN allows the system to group related content without predefining the number of clusters. In addition, the large language model helps transform content clusters into more understandable information for users, including topics, summaries, sentiment, risk levels, and content types.

V. CONCLUSION AND FUTURE WORK

This paper proposed a framework for near-real-time social trend detection and analysis from Vietnamese online content. The system combines Kafka, PySpark, embedding models, HDBSCAN, and large language models to collect, process, cluster, and interpret trends from online data. Experiments on two data batches with 5,305 input items showed that the system retained 4,679 items after quality filtering, detected 25 trend clusters with an average cluster size of about 147 items, and achieved 93.3% trend correctness in a preliminary manual evaluation on 15 trends. These results suggest that combining semantic representation, density-based clustering, and LLM-based analysis is a feasible approach for Vietnamese social trend detection.

However, the current experiment still has several limitations. The detected trend data mainly came from YouTube and TikTok, while News did not contribute any item to the detected clusters. Therefore, the current results should be considered an initial validation on video-based platforms rather than complete evidence of multi-source effectiveness. In future work, we plan to expand and balance the dataset with more news sources, improve RSS-based data collection, and design source-specific preprocessing strategies for online news. We also plan to conduct full quantitative comparisons with baseline methods, including keyword-based detection, K-means, DBSCAN, LDA, BERTopic, and HDBSCAN without LLM. Additional evaluation metrics such as precision, recall, F1-score, clustering quality, topic coherence, and duplicate detection accuracy will be considered. Moreover, future manual evaluation should involve more independent annotators and report inter-annotator agreement. Finally, system-level scalability evaluation, including throughput, batch processing latency, memory usage, and Spark execution time, will be conducted to better validate the near-real-time capability of the framework.

ACKNOWLEDGMENT

This research was supported by the University of Information Technology, Vietnam National University, Ho Chi Minh City, Vietnam.

REFERENCES

[1] DataReportal, "Digital 2025: Vietnam," https://datareportal.com/reports/digital-2025-vietnam, 2025, accessed: 2026-05-17.
[2] J. Allan, "Introduction to topic detection and tracking," in *Topic Detection and Tracking: Event-Based Information Organization*. Springer, 2002.
[3] M. Cataldi, L. Di Caro, and C. Schifanella, "Emerging topic detection on twitter based on temporal and social terms evaluation," in *Proceedings of the 10th International Workshop on Multimedia Data Mining*, 2010.
[4] L. M. Aiello, G. Petkos, C. Martin, D. Corney, S. Papadopoulos, R. Skraba, A. Goker, Y. Kompatsiaris, and A. Jaimes, "Sensing trending topics in twitter," *IEEE Transactions on Multimedia*, vol. 15, no. 6, pp. 1268–1282, 2013.
[5] B. Batrinca and P. C. Treleaven, "Social media analytics: A survey of techniques, tools and platforms," *AI & Society*, vol. 30, pp. 89–116, 2015.
[6] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," in *Proceedings of EMNLP-IJCNLP*, 2019.
[7] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proceedings of the Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2013, pp. 160–172.
[8] W. Zhang, Y. Deng, B. Liu, S. J. Pan, and L. Bing, "Sentiment analysis in the era of large language models: A reality check," in *Findings of NAACL*, 2024.
[9] J. Kleinberg, "Bursty and hierarchical structure in streams," *Data Mining and Knowledge Discovery*, vol. 7, pp. 373–397, 2003.
[10] A. Abdelrazek, Y. Eid, E. Gawish, W. Medhat, and A. Hassan, "Topic modeling algorithms and applications: A survey," *Information Systems*, 2023.
[11] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering," *Journal of Open Source Software*, vol. 2, no. 11, p. 205, 2017.
[12] M. Grootendorst, "Bertopic: Neural topic modeling with a class-based tf-idf procedure," 2022.
[13] H. Jiang *et al.*, "Topic detection and tracking with time-aware document embeddings," in *Proceedings of LREC-COLING*, 2024.
[14] D. Hanny *et al.*, "Clustering-based joint topic-sentiment modeling of social media short texts," *Information*, vol. 15, no. 4, 2024.
[15] Apache Kafka, "Apache Kafka Documentation," https://kafka.apache.org/documentation/, 2026, accessed: 2026-05-17.
[16] Apache Spark, "PySpark Documentation," https://spark.apache.org/docs/latest/api/python/, 2026, accessed: 2026-05-17.
[17] MongoDB, "MongoDB Documentation," https://www.mongodb.com/docs/, 2026, accessed: 2026-05-17.